

TP d'Algorithmique

N°3

Table des matières :

- Exercice 1 :
 - Description :
 - Jeux d'essais
- Exercice 2 :
 - Description :
 - Jeux d'essais
- Exercice 3 :
 - Description :
 - Jeux d'essais

Exercice 1 : Calculatrice

```
if __name__ == "__main__" :
    operande1 : float
    operande2 : float
    resultat : float
    operateur : str
    operande1 = float(input("Entrez un premier nombre réel : "))
    operateur = str(input("Entrez un opérateur : "))
    operande2 = float(input("Entrez un autre nombre réel : "))
    if operateur == "+" :
        resultat = operande1 + operande2
        print("Le résultat est : ", resultat)
    if operateur == "-" :
        resultat = operande1 - operande2
        print("Le résultat est : ", resultat)
    if operateur == "*" :
        resultat = operande1 * operande2
        print("Le résultat est : ", resultat)
    if operateur == "/" :
        if operande2 == 0 :
            print("Erreur, la division par zéro est impossible")
        else :
            resultat = operande1 / operande2
            print("Le résultat est : ", resultat)
    if operateur != "+" and operateur != "-" and operateur != "*" and operateur != "/" :
        print("Erreur, opérateur inconnu")
```

Ce programme a pour objectif de servir de calculatrice en entrant 2 nombres réels ainsi qu'un opérateur parmi « +,-,*,/ » et donc en faisant le calcul des deux nombres avec l'opérateur saisi.

Jeux d'essais :

```
Entrez un premier nombre réel : 5
Entrez un opérateur : *
Entrez un autre nombre réel : 8
Le résultat est : 40.0
```

Si l'on rentre deux nombres entiers positifs cela fonctionne correctement

```
Entrez un premier nombre réel : 6
Entrez un opérateur : -
Entrez un autre nombre réel : -7
Le résultat est : 13.0
```

Si l'on rentre un positif ainsi qu'un négatif en deuxième le double '-' est bien interprété par python

```
Entrez un premier nombre réel : -5  
Entrez un opérateur : *  
Entrez un autre nombre réel : -9  
Le résultat est : 45.0
```

Avec deux nombres négatifs ils sont aussi correctement traités par python

```
Entrez un premier nombre réel : 5  
Entrez un opérateur : /  
Entrez un autre nombre réel : 8  
Le résultat est : 0.625
```

La division fonctionne tout aussi bien sauf :

```
Entrez un premier nombre réel : 5  
Entrez un opérateur : /  
Entrez un autre nombre réel : 0  
Erreur, la division par zéro est impossible
```

Si l'on divise par zéro cela affiche un message d'erreur comme quoi on ne peut pas diviser par zéro

```
Entrez un premier nombre réel : 5  
Entrez un opérateur : )  
Entrez un autre nombre réel : 7  
Erreur, opérateur inconnu
```

Si l'on saisit n'importe quel autre caractère que les opérateurs écrit dans la partie description du programme cela affiche une erreur comme quoi l'opérateur est inconnu

Exercice 2 : PGCD

```
if __name__ == "__main__" :  
    a : float  
    b : float  
  
    a = float(input("Entrez un nombre : "))  
    b = float(input("Entrez un autre nombre : "))  
    while a != int(a) or b != int(b) :  
        print("Erreur, les nombres doivent être des entiers")  
        a = float(input("Entrez un nombre ENTIER: "))  
        b = float(input("Entrez un autre nombre ENTIER : "))  
    while a <= 0 or b <= 0 :  
        print("Erreur, les nombres ne peuvent pas être nuls ou négatifs")  
        a = float(input("Entrez un nombre POSITIF et NON NUL : "))  
        b = float(input("Entrez un autre nombre POSITIF NON NUL : "))  
    while a != b :  
        if a > b :  
            a = a - b  
        else :  
            b = b - a  
    print("Le PGCD est : ", a)
```

Ce programme utilise donc l'algorithme d'Euclide original (les soustractions successives) pour connaître le plus grand diviseur commun ou PGCD. Un premier while pour vérifier si les nombres saisis sont entiers s'ils ne le sont pas on doit ressaisir les deux nombres. Ensuite un deuxième while pour vérifier s'ils ne sont pas négatifs ou nul et donc enfin après les 2 conditions on applique l'algorithme d'Euclide.

Jeux d'essais :

```
Entrez un nombre : 1.8  
Entrez un autre nombre : 3  
Erreur, les nombres doivent être des entiers  
Entrez un nombre ENTIER: 0  
Entrez un autre nombre ENTIER : 4  
Erreur, les nombres ne peuvent pas être nuls ou négatifs  
Entrez un nombre POSITIF et NON NUL : 35  
Entrez un autre nombre POSITIF NON NUL : 3  
Le PGCD est : 1.0
```

Si l'on saisit un nombre décimal on doit donc ressaisir, de la même façon si l'on saisit un nombre négatif ou nul.

Exercice 3 : Date

```
if __name__ == "__main__" :  
    jour : float  
    mois : float  
    annee : float  
    bissextile : bool  
    bissextile = False  
    jour = float(input("Entrez le jour : "))  
    mois = float(input("Entrez le mois : "))  
    annee = float(input("Entrez l'année : "))  
  
    while jour != int(jour) or mois != int(mois) or annee != int(annee) :  
        print("Erreur, les nombres doivent être des entiers")  
        jour = float(input("Entrez le jour : "))  
        mois = float(input("Entrez le mois : "))  
        annee = float(input("Entrez l'année : "))  
    while jour < 1 or jour > 31 :  
        print("Erreur, le jour doit être compris entre 1 et 31")  
        jour = int(input("Entrez le jour : "))  
    while mois < 1 or mois > 12 :  
        print("Erreur, le mois doit être compris entre 1 et 12")  
        mois = int(input("Entrez le mois : "))
```

Cette partie du programme permet d'initialiser et de vérifier la validité des valeurs des variables.

Jeux d'essais :

```
Entrez le jour : 13.3  
Entrez le mois : 12  
Entrez l'année : 2000  
Erreur, les nombres doivent être des entiers
```

```
Entrez le jour : 32  
Entrez le mois : 12  
Entrez l'année : 2000  
Erreur, le jour doit être compris entre 1 et 31
```

```
Entrez le jour : 31  
Entrez le mois : 13  
Entrez l'année : 2000  
Erreur, le mois doit être compris entre 1 et 12
```

On vérifie donc ici que chaque valeur est bien valide pour une date

```
if annee % 4 == 0 :
    if annee % 100 != 0 :
        bissextile = True
    else :
        if annee % 400 == 0 :
            bissextile = True
if mois == 2 :
    if bissextile :
        if jour > 29 :
            print("Erreur, le mois de février ne peut pas avoir plus de 29 jours")
        else :
            print("La date est valide")
    else :
        if jour > 28 :
            print("Erreur, le mois de février ne peut pas avoir plus de 28 jours")
        else :
            print("La date est valide")
else :
    if mois == 4 or mois == 6 or mois == 9 or mois == 11 :
        if jour > 30 :
            print("Erreur, ce mois ne peut pas avoir plus de 30 jours")
        else :
            print("La date est valide")
    else :
        if jour > 31 :
            print("Erreur, ce mois ne peut pas avoir plus de 31 jours")
        else :
            print("La date est valide")
```

Cette autre partie sert donc à vérifier la validité totale de la date saisie donc l'objectif principal du programme.

Jeux d'essais :

```
Entrez le jour : 32
Entrez le mois : 13
Entrez l'année : 2000
Erreur, le jour doit être compris entre 1 et 31
Entrez le jour : -12
Erreur, le jour doit être compris entre 1 et 31
Entrez le jour : 12
Erreur, le mois doit être compris entre 1 et 12
Entrez le mois : -15
Erreur, le mois doit être compris entre 1 et 12
Entrez le mois : 3
La date est valide
```

Dans ce test on écrit des dates invalides sous toutes les formes jusqu'à obtenir une date valide.

```
Entrez le jour : 31
Entrez le mois : 2
Entrez l'année : 2000
Erreur, le mois de février ne peut pas avoir plus de 29 jours
```

```
Entrez le jour : 29
Entrez le mois : 2
Entrez l'année : 2001
Erreur, le mois de février ne peut pas avoir plus de 28 jours
```

Ceci montre donc les 2 erreurs avec en premier une année bissextile (2000) donc que février ne peut avoir plus de 29 jours et le deuxième avec une année non bissextile (2001) donc que février ne peut avoir plus de 28 jours.

```
Entrez le jour : 31
Entrez le mois : 4
Entrez l'année : 2000
Erreur, ce mois ne peut pas avoir plus de 30 jours
```

```
Entrez le jour : 31
Entrez le mois : 6
Entrez l'année : 2000
Erreur, ce mois ne peut pas avoir plus de 30 jours
```

```
Entrez le jour : 31
Entrez le mois : 9
Entrez l'année : 2000
Erreur, ce mois ne peut pas avoir plus de 30 jours
```

```
Entrez le jour : 31
Entrez le mois : 11
Entrez l'année : 2000
Erreur, ce mois ne peut pas avoir plus de 30 jours
```

Ce test montre qu'effectivement on ne peut pas avoir 31 jours dans les mois d'avril, juin, septembre et novembre.

